

```
nginx
```

[Copy code](#)

```
IPv4 Address: 192.168.1.50
IPv4 Address: 10.0.0.25
IPv4 Address: 172.16.0.40
```

Once you have a LAN IP:

Try:

```
cpp
```

[Copy code](#)

```
http://YOUR_LAN_IP:8000
```

✓ Summary of what you need to do

1 Connect to OFFICE WiFi

(No Hotspot, No Bluetooth, No Ethernet disconnected)

2 Run `ipconfig` again

Find IPv4 under the WiFi adapter.

3 Use the correct format:

```
cpp
```

[Copy code](#)

```
http://<your-ip>:8000
```

If you want, send me your new ipconfig after connecting to office WiFi, and I will tell you the exact URL to use.

📄 📌 🗑️ ⬆️ ↺ ⋮

okay it works but is there a way on ensuring that this this system is always running on some computer so the team can access it

Yes — you can make your system run automatically 24/7 on a specific computer in the office, even if nobody is logged in. It can behave like a real server.

Here are the 3 best ways to host your FastAPI + Automation service permanently:

✓ OPTION 1: Run as a Windows Service (Best Option)

This is the most professional and reliable method.

When you turn your script into a Windows Service:

- ✓ It starts automatically when the PC boots
- ✓ It runs even if no user is logged in
- ✓ It restarts automatically if it crashes
- ✓ Other people can always access the app
- ✓ Automation service always runs in background

You will use NSSM (Non-Sucking Service Manager) — easy and stable.

★ STEP 1 — Install NSSM

Download NSSM:

<https://nssm.cc/download> ↗

Extract it somewhere, e.g.:

```
makefile
```

[Copy code](#)

```
C:\nssm\
```

★ STEP 2 — Register your FastAPI app as a service

Open Command Prompt (as Admin):

```
makefile
```

[Copy code](#)

```
C:\nssm\nssm.exe install DHLTrackingSystem
```

A window appears. Fill in:

Application tab:

Path:

```
makefile  
  
C:\Users\Asithandle\Downloads\dhl-tracking-system\venv\Scripts\python.exe
```

Startup directory:

```
makefile  
  
C:\Users\Asithandle\Downloads\dhl-tracking-system\
```

Arguments:

```
arduino  
  
run.py
```

Details tab:

Display name:

```
sql  
  
DHL Tracking System
```

Exit tab:

Choose:

```
makefile  
  
Restart: Service
```

★ STEP 3 — Start the service

```
powershell  
  
nssm start DHLTrackingSystem
```

Now the application is permanently running in the background.

★ STEP 4 — Test

On another PC:

```
cpp  
  
http://<server-ip>:8000
```

It will work even if:

- You log out
- You close the terminal
- You close everything
- The PC reboots

⚡ OPTION 2: Auto-start on login (easiest, but less reliable)

Create a `.bat` file:

`start_server.bat`

```
bat  
  
cd C:\Users\Asithandle\Downloads\dhl-tracking-system  
call venv\Scripts\activate  
py run.py
```

Then put it inside:

```
makefile  
  
shell:startup
```

Downside:

- ❌ Only runs when user logs in
- ❌ Stops if laptop sleeps
- ❌ Not reliable for 24/7 automation

✖ Not reliable for 24/7 automation

🔥 OPTION 3: Deploy on a small office server / mini PC

If your office has a spare:

- Dell Optiplex
- HP workstation
- Intel NUC
- Mini PC
- Server machine

Install Windows or Linux, then run your system permanently on that machine.

It becomes your **internal office API server**.

Advantages:

- ✓ Always running
- ✓ Not tied to your laptop
- ✓ Fast & stable
- ✓ Professional setup

This is what most offices do for internal systems.

🏆 Best Recommendation for You

Since this is an internal office tool:

👉 Use Option 1: Create a Windows Service with NSSM

This gives you:

- 24/7 uptime
- Automatic restarts
- Hardware independence from your laptop
- Professional deployment
- Automation service always working

📌 If you want, I can prepare:

- ✓ A ready-to-run Windows Service installer
- ✓ A `.bat` script for installing the service
- ✓ A monitoring dashboard `/system/status`
- ✓ A log viewer UI
- ✓ A service restart endpoint

Just tell me what you prefer.

📄 📌 🗨️ ⬆️ ↺️ ⋮

+ Ask anything



ChatGPT can make mistakes. Check important info. See [Cookie Preferences](#).